

その契約、証明できますか？

並列AIエージェント開発のための「形式契約」

formal-agent-contracts — Claude Code プラグイン

IID Systems / @kotaroyamame

AI駆動開発勉強会 LT

並列エージェント開発、こうなりませんか

- エージェントA が作った API を、エージェントB が微妙に違う形で呼ぶ
- CLAUDE.md に書いた規約が、受け渡しのたびに少しずつ無視される
- 生成速度 >> レビュー速度。人間の目が最後の砦になっている

制約は主張しただけでは維持されない — エージェント間の受け渡しで劣化する

(いわゆる constraint drift — arXiv:2605.10481, 2026)

仕様駆動開発は「半分」解決した

Kiro / Spec Kit / cc-sdd — 仕様を先に書く文化は定着しつつある 🎉

でも `spec.md` は散文 (Markdown) :

	人間が読める	エージェントが読める	機械が検証できる
spec.md (散文)	✓	✓	✗
型定義	✓	✓	型までは ✓
形式仕様 (契約)	✓	✓	✓

「実装が仕様を満たしているか」を判定するのは、いまだに LLM の自己申告と人間のレビュー

提案：エージェント間の契約を「形式仕様」で書く

VDM-SL — ISO標準の形式仕様記述言語

- 型不変条件・事前条件・事後条件を機械が判定できる形で書ける
- VDMJ が構文・型を機械チェックし、証明責務（PO）を自動生成
- Z3 で PO を自動証明（または反例を発見）

「でも形式手法って難しいんでしょ？」

→ **書くのは Claude。人間は日本語で要件を言うだけ**

デモ①：日本語 → 形式契約

入力（これだけ）：

タスク管理エージェントを定義して。

- タスクには ID・タイトル・状態 (Todo/InProgress/Done) がある
- Done になったタスクは Todo/InProgress に戻せない
- タイトルは100文字以内

出力（VDM-SL、抜粋・簡略化）：

```
Task :: id : TaskId  title : seq1 of char  status : Status
inv t == len t.title <= 100;

ValidTransition: Status * Status -> bool
ValidTransition(fromSt, toSt) ==
  if fromSt = toSt then true
  else if fromSt = <Done> then false    -- Done からは戻れない
  else true;
```

デモ②：機械が検証する（実測出力）

> 仕様を検証して

Parsed 1 module in 0.029 secs. No syntax errors

Type checked 1 module in 0.071 secs. No type errors

Generated 38 proof obligations

> qc (QuickCheck で全 P0 を判定)

P0 #8 初期状態は不変条件を満たすか → PROVABLE  (witness: 空ボード, nextId=1)

P0 #22 CreateTask の事後条件 → FAILED  反例つき

反例: nextId=1 のとき RESULT=1 ≠ nextId~ - 1 (=0)

→ 事後条件の off-by-one (nextId~ の取り違い)

- この反例、**同梱サンプルに実在したバグ**です（この発表の準備中に検出 → 修正済み。

再現は修正前コミット d6427bc で。修正後は FAILED 0 / PO 37個)

- PO は QuickCheck / Z3 が自動処理。人間が見るのは反例が出たときだけ
- LLM の「たぶん大丈夫です」ではなく、**チェッカーの判定**

デモ③：契約からテストとコードが出てくる

```
// generate-tests が生成する Vitest 契約テスト (イメージ)
it("rejects Done -> Todo transition", () => {
  expect(() => mgr.changeStatus(doneTaskId, "Todo"))
    .toThrow(ContractError); // 事前条件違反
});
```

- タイトル100文字超の拒否など、境界値テストも同時に生成される
- TS/Python スキャフォールドにはランタイム契約チェックが埋め込まれる
- 別のエージェントが契約を破るコードを書いたら、テストが落ちて止まる
- コードが先に変わったら reconcile-code で契約と照合 — 乖離は放置されない

つまり：ガードレールから「証明できるガードレール」へ

「AIエージェント時代に、TDD は『ガードレール』になる」

(t_wada × やっとむ対談, Agile Journey, 2025)

その延長線上に——

レイヤー	担保するもの	誰が判定
spec.md	意図の共有	人間 + LLM
テスト (TDD)	特定ケースの正しさ	テストランナー
形式契約	全ケースの整合性	VDMJ + Z3

型は形しか守れず、テストは書いたケースしか守らない。契約は **1つのソースから型・テスト・ランタイムチェックを全部生成** — エージェントが増えるほど単一ソースの価値が上がる

ちなみに VDM は「日本で一番実績のある」形式手法

- モバイルFeliCa ICチップ（おサイフケータイ／モバイルSuica 搭載）のファームウェア仕様は VDM++ で記述された
 - スマホをかざして改札を通るたび、「形式手法で書かれた仕様」が動いている
- NII（TopSE）・九州大などに教育基盤、日本語文献も豊富

そして 2026年7月現在、調べた限り VDM × AIエージェントの公開ツールはこれが唯一

（GitHub 公開リポジトリを調査。TLA+/Lean/Dafny には既にエージェント統合があるが、VDM では見つからなかった）

今すぐ試せます

```
/plugin marketplace add anthropics/claude-plugins-community  
/plugin install formal-agent-contracts@claude-community
```

インストールしたら、これを打つだけ：

タスク管理エージェントの契約を定義して

既存リポジトリなら「このコードから仕様を抽出して」の一言でも動きます

契約定義とテスト生成は外部ツール不要（VDMJ/Z3 はフル検証時のみ）

github.com/kotaroyamame/formal-agent-contracts | スキルドキュメント日英対応 / MIT

形式手法 × AIコーディング、日本からやっていきましょう

Appendix（時間があれば / 質疑用）

Appendix: 13スキルの全体像

フォワード（仕様 → コード）

define-contract → verify-spec → smt-verify → generate-code → generate-tests

（一括実行: integrated-workflow）

リバース（既存コード → 仕様）

extract-spec → refine-spec → reconcile-code

（一括実行: reverse-workflow）

入出力: import-natural-spec / export-human-spec (Markdown $\rightleftharpoons$ VDM-SL)

参照: formal-methods-guide (VDM-SL の文法・PO の意味を解説)

Appendix: 探索的評価（正直な数字）

3課題 × 5試行を、プラグインあり/なしの2条件で比較（計30試行）：

メトリクス	Δ	効果量 (Cliff's δ)
仕様カバレッジ	+42.8pp	0.74 (large)
テスト有効性	+20.3pp	0.56 (large)

⚠ 限界：同一モデルが両群を生成 / キーワードベース採点 / 有意性検定は未実施。

テスト有効性はテストコードのヒューリスティック分析による代替計測で、追加のミュートーションテストでは両群とも kill rate 100%（天井効果）のため群間差は未検出。

探索的な結果であり、独立した追試を計画中。詳細は [eval/results/report.md](#)

Appendix: よくある質問

Q. Anthropic の Outcomes (Managed Agents のループリック採点) と何が違う？

A. Outcomes は LLM による確率的な採点。こちらは VDMJ/Z3 による決定的・再現可能な判定。併用できます。

Q. VDM-SL を読めないとレビューできないのでは？

A. export-human-spec が自然言語の仕様書を生成します。逆方向 (Markdown → VDM-SL) も import-natural-spec で。

Q. 学習コストは？

A. 契約は Claude が書き、PO の意味も日本語で解説させられます。読めるようになるのは使いながらで十分。